Partitioned Hash Join with Duplicates
Module 2: From Partition Mapping to Full Join

Alessandro Mariani 654391

1 Algorithm Overview

The partitioned hash join pipeline is structured as a sequence of five major phases: histogram computation, prefix sum and offset building, scatter (partition redistribution), local join, and result accumulation. Both relations R and S go through the first three phases before the join is performed independently on each partition.

From a parallelization perspective, histogram, scatter, and local join are the phases with enough independent work to benefit from multi-threading: histogram and scatter expose per-record parallelism, while the local join exposes per-partition parallelism after data redistribution. By contrast, prefix-sum, offset construction, and final accumulation are kept sequential because their work is only $O(P)$, $O(T \cdot P)$, and $O(T)$, respectively, so the added synchronization needed to parallelize them would be less attractive than their already small absolute cost.

The partition mapping function from Module 1 is reused unchanged. Given a 64-bit key k, the low and high 32-bit halves are XOR-folded and multiplied by the Fibonacci constant $A = \texttt{0x9E3779B9}$, yielding

$$h(k,p) = \left((k_{\text{low}} \oplus k_{\text{high}}) \cdot A\right) \bmod 2^{32} \gg (32-p), \quad (1)$$

where $P = 2^p$. This maps each key deterministically to one of P partitions using only XOR, a 32-bit multiply, and a shift.

2 Parallelization Strategy

The remaining design problem is therefore how to parallelize the profitable phases without introducing unnecessary synchronization, contention on shared structures, or severe load imbalance under skew.

2.1 Thread lifecycle and barrier

Threads are created *once* and reused across all phases via `std::barrier`. Spawning and joining threads once per phase would introduce $O(T)$ serial overhead that would dominate at high thread counts. The barrier's *completion function* is executed once per synchronization point by the last arriving thread and performs all inter-phase bookkeeping. The main thread is not kept idle as a pure coordinator: after spawning $T-1$ workers it also executes worker 0, so all available workers participate in useful computation while the barrier still provides a single, well-defined synchronization point between phases.

2.2 Histogram (parallel)

Each of the T threads owns a private local histogram of size P. Thread t scans its block $[tN/T,\ (t+1)N/T)$ of the input and increments its own histogram. Because each histogram is a separate `vector` on the heap, there is no false sharing and no synchronization inside this phase. This choice spends $O(T \cdot P)$ extra memory in exchange for removing atomics or locks from the hot loop, which is preferable because the histogram update is executed once per input record and would otherwise become highly contended.

2.3 Prefix sum and offset build (sequential)

Once all histograms are ready, the barrier completion function merges the T local histograms into a global histogram in $O(T \cdot P)$ time, computes an exclusive prefix sum to obtain global partition begin offsets, and derives per-thread scatter cursors so that thread t writes records for partition p starting at

$$\text{global\_begin}[p] + \sum_{j<t} \text{lh}_j[p]. \quad (2)$$

With $P \leq 1024$ and $T \leq 32$, the merge is at most $\approx 32\,000$ additions and takes $\ll 1$ ms in isolation. However, the `data_out.resize(N)` buffer allocation also occurs here, making this the dominant serial section. Keeping merge, prefix, and offset construction in one sequential barrier-completion step is therefore a deliberate trade-off: the implementation accepts a small serialized section to avoid adding more synchronization and orchestration complexity to work that is itself computationally small.

2.4 Scatter (parallel, lock-free)

Each thread copies its block from the input to the partitioned output buffer using its private cursor copy. No two threads write to the same location by construction, so there are no locks or atomics. In practice this also keeps false sharing negligible, since each thread advances independently through disjoint output regions. The price of this lock-free scatter is the precomputation of per-thread offsets, but this is a good trade because it turns potentially conflicting writes into deterministic disjoint output ranges.

2.5 Local join (cyclic over partitions)

After both relations are partitioned, thread t handles partitions t, $t+T$, $t+2T, \ldots$ (cyclic distribution). Cyclic assignment amortizes partition-size skew better than block assignment: with a skewed key distribution a single large partition would otherwise serialise the join. Inside each partition, a local `unordered_map` counts key multiplicities in R; then S is scanned to emit matches. Each thread accumulates its output into a private padded result structure, avoiding false sharing during the final per-thread updates. Dynamic scheduling was not used here: a shared queue or atomic work counter would add coordination overhead, while cyclic static assignment captures most of the load-balancing benefit with essentially zero runtime synchronization.

2.6 Accumulation (sequential)

Thread results (`join_count`, two 64-bit checksums) are reduced sequentially. With $T \leq 32$ the cost is negligible, so a parallel reduction would add complexity and an extra synchronization step for very little benefit.

3 Correctness Verification

The sequential version provides the reference output for any given input seed. Correctness of the parallel version is verified by comparing `join_count`, `checksum1`, and `checksum2` against the sequential baseline. For very small inputs ($N_R \leq 500$ and $N_S \leq 500$), an additional naive $O(N_R N_S)$ verifier is executed outside the timed region. For larger inputs, the comparison against the sequential baseline through `join_count` and the two checksums is sufficient and does not perturb the measured execution time.

4 Experimental Setup

All experiments were run on `node01` of the `spmcluster` (`normal` partition), which provides 32 physical cores. The binary was compiled with `g++ -O3 -march=native`. Each configuration was repeated 5 times (plus 1 warm-up run) and the *median* execution time is reported:

$$t_{\text{med}} = \text{median}\big(t^{(1)}, t^{(2)}, t^{(3)}, t^{(4)}, t^{(5)}\big). \tag{3}$$

The sequential baseline (`hashjoin_seq`) uses the same input generation and mapping function.

5 Performance Results

5.1 Strong Scaling

Table 1 and Figure 1 report results for the larger of the two strong-scaling configurations ($N_R = 20$M, $N_S = 40$M, $P = 512$). The sequential baseline takes $t_{\text{seq}} = 2.355$ s. The reported metrics are computed as

$$\text{Speedup}(T) = \frac{t_{\text{seq}}}{t_T}, \tag{4}$$

$$\text{Efficiency}(T) = \frac{\text{Speedup}(T)}{T} = \frac{t_{\text{seq}}}{T \cdot t_T}, \tag{5}$$

where t_T is the median parallel execution time with T threads.

T	Time (ms)	Speedup	Efficiency
1	2317.9	1.02	1.02
2	1280.7	1.84	0.92
4	828.2	2.84	0.71
8	505.9	4.65	0.58
12	394.5	5.97	0.50
16	337.3	6.98	0.44
20	363.2	6.48	0.32
22	350.3	6.72	0.31
24	333.4	7.06	0.29
26	322.0	7.31	0.28
28	310.9	7.57	0.27
30	305.5	7.71	0.26
32	294.1	8.01	0.25

Table 1: Strong scaling: $N_R = 20\text{M}$, $N_S = 40\text{M}$, $P = 512$, $t_{\text{seq}} = 2355\,\text{ms}$. Median of 5 runs on node01.

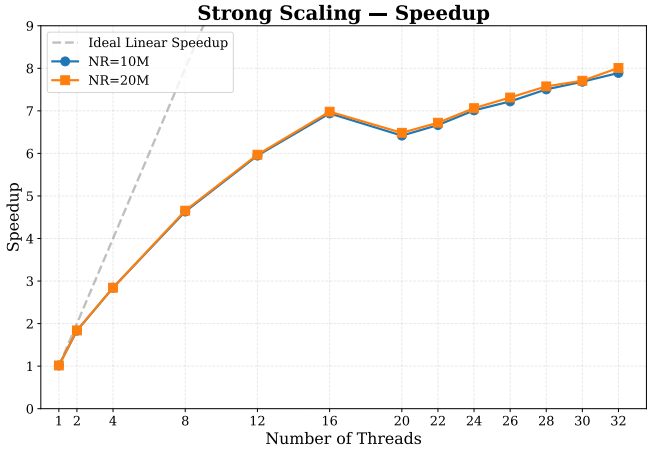

Figure 1: Observed speedup vs. ideal linear speedup. The curve shows strong gains up to about 16 threads, followed by clear diminishing returns.

The strong-scaling results indicate that the implementation benefits substantially from parallelism up to about $T = 16$, after which the marginal gain from additional threads becomes much smaller. The best result is **8.01**× at $T = 32$ (25% efficiency), which confirms that the parallel phases remain effective, but are increasingly limited by a non-negligible serial component. The value slightly above 1× at $T = 1$ is best interpreted as measurement noise rather than genuine superlinear behaviour.

The dip at $T = 20$ (6.48×) is compatible with NUMA-related effects: `node01` is a dual-socket machine, so part of the working set may cross a socket boundary once the thread count exceeds 16. This explanation is plausible given the memory-intensive scatter and partitioning phases, although it is not directly validated by hardware-counter measurements in the present study.

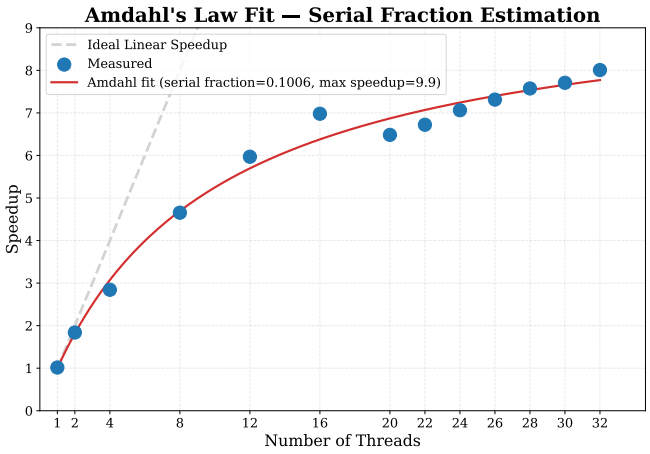

Figure 2: Amdahl fit: serial fraction $f = 0.101$, $S_{\max} = 9.94\times$.

5.2 Amdahl Analysis

Fitting Amdahl's law to the observed speedup points gives

$$S(T) = \frac{1}{f + (1-f)/T}, \tag{6}$$

which yields a serial fraction $f \approx 0.101$ and a theoretical maximum speedup

$$S_{\max} = \frac{1}{f} \approx 9.94\times. \tag{7}$$

Figure 2 shows the fit. The estimate is consistent with the flattening already visible in Figure 1: once the thread count is high enough, the serialized portion of the pipeline dominates the remaining execution time, so adding more workers yields only limited additional speedup. The main contributors to this serialized work are the barrier-completion tasks: histogram merge ($O(T \cdot P)$), global prefix sum ($O(P)$), per-thread offset build ($O(T \cdot P)$), and output-buffer allocation (`Rdata/Sdata.resize(N)`).

5.3 Phase Breakdown

Figure 3 shows the time breakdown per phase at $T = 32$ compared with $T = 1$. The scatter phase scales well (roughly 16× reduction from $T = 1$ to $T = 32$ for relation R), and the join phase

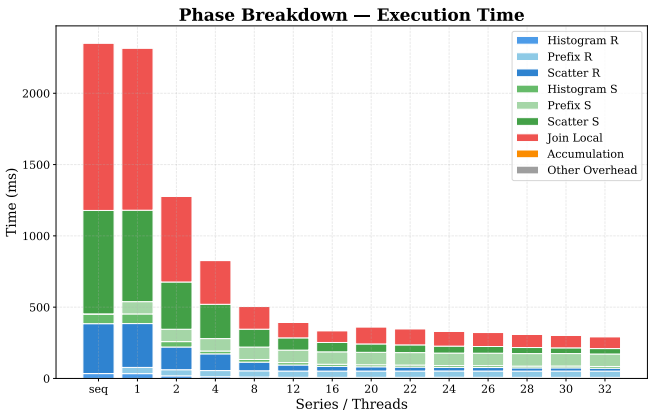

Figure 3: Phase-level time breakdown. "Prefix" includes the serial barrier completion (merge, offset build, and buffer allocation).

also shrinks substantially thanks to cyclic partition distribution. By contrast, the component labelled "Prefix" remains nearly constant across thread counts. This should not be read as evidence that the prefix-sum arithmetic itself is expensive; rather, the measurement includes the entire barrier completion, which also contains offset construction and the $O(N)$ output-buffer allocation. The phase breakdown therefore identifies the serialized pipeline orchestration work, not the prefix-sum kernel in isolation, as the main bottleneck.

5.4 Weak Scaling

The weak scaling experiment keeps the per-thread workload constant at $N_R^{\text{base}} = 500\,000$, $N_S^{\text{base}} = 1\,000\,000$ ($P = 512$). Table 2 and Figure 4 report the Weak Scaling Efficiency defined as

$$\text{WSE}(T) = \frac{t_1}{t_T}. \tag{8}$$

WSE degrades from 1.000 at $T = 1$ to 0.345 at $T = 32$, while the median execution time increases from 81.5 ms to 236.4 ms. Figure 4 shows that the drop is already visible up to 16 threads and becomes steeper after 20 threads. This mirrors the strong-scaling results: the same serialized barrier-completion work that limits the speedup at large T becomes even more visible here because the total problem size grows with the number of threads. In particular, `resize(N)` scales with the global input size, while

T	N_R total	Time (ms)	WSE
1	500 K	81.5	1.000
2	1 000 K	88.6	0.920
4	2 000 K	101.5	0.803
8	4 000 K	111.8	0.729
12	6 000 K	125.4	0.650
16	8 000 K	139.9	0.583
20	10 000 K	186.8	0.436
22	11 000 K	195.8	0.416
24	12 000 K	203.8	0.400
26	13 000 K	209.8	0.388
28	14 000 K	218.9	0.372
30	15 000 K	226.5	0.360
32	16 000 K	236.4	0.345

Table 2: Weak scaling efficiency ($P = 512$, $N_R^{\text{base}} = 500$ K per thread).

the useful compute per thread is held constant. As a result, the serial fraction *increases* with problem size, which is the main cause of the observed weak-scaling degradation.

5.5 Duplicate Density

The duplicate-density experiment varies `max_key` from 10^2 (highly skewed, few distinct keys) to 10^7 (nearly unique keys) with $T = 32$, $P = 512$, $N_R = 10$ M. Figure 5 shows that the intermediate range `max_key` $\in [10^3, 10^5]$ remains close to 8×, peaking at 8.06× for `max_key` $= 10^4$. The two extremes reveal different limiting regimes. With very high duplicate rates (`max_key` $= 10^2$), speedup drops to 6.06× because a small number of hot keys creates partition imbalance and therefore poorer join load balance. At the opposite extreme, the near-unique case (`max_key` $= 10^7 = N_R$) reaches 9.51×: here the sequential time grows sharply as the `unordered_map` working set expands, whereas the parallel version still benefits from smaller partition-local working sets. The result is therefore not only a change in speedup, but also a shift in the dominant cost from imbalance under heavy duplication to memory pressure in the near-unique regime.

5.6 Partition Sensitivity

Figure 6 shows the effect of P on parallel execution time at $T = 32$, $N_R = 10$ M. Performance improves monotonically from $P = 16$ (296.4 ms) to $P = 1024$ (146.9 ms), indicating that a coarse partitioning leaves too much work and too large a work-

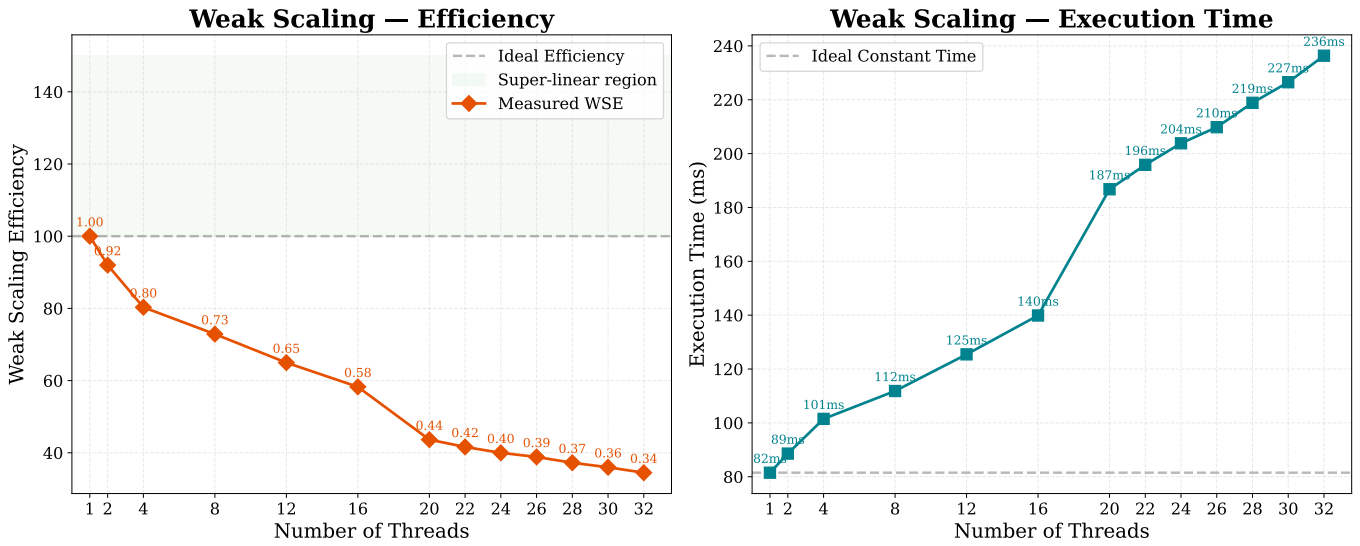

Figure 4: Weak scaling for fixed per-thread workload: execution time and efficiency vs. thread count.

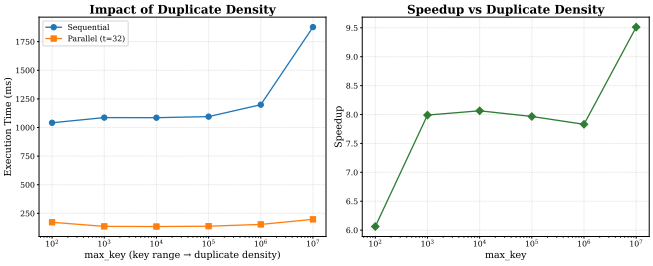

Figure 5: Effect of key-space size on speedup at $T = 32$.

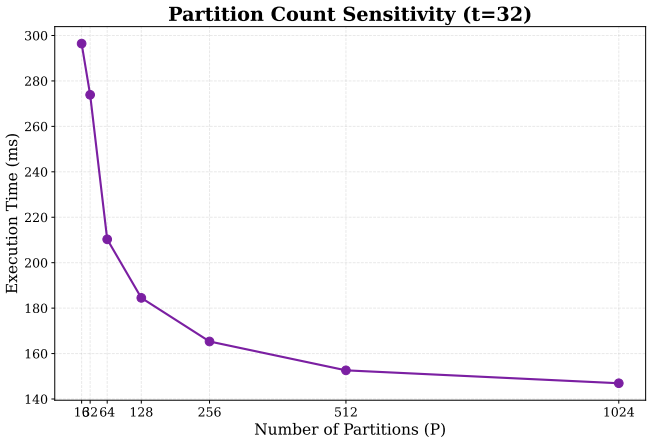

Figure 6: Parallel execution time vs. P at $T = 32$.

ing set inside each local join. Increasing P improves cache locality and usually balances the work better across threads, but it also enlarges the histogram and scatter bookkeeping structures. The result is a trade-off rather than an unlimited gain. In these experiments the benefit saturates near $P = 1024$: moving from $P = 512$ to $P = 1024$ only improves execution time from 152.6 ms to 146.9 ms, so further doubling P would likely increase histogram/scatter overhead without proportional locality benefits.

6 Discussion

Overall, the results indicate that the algorithmic decomposition is sound: the histogram, scatter, and local-join phases all benefit clearly from parallel execution, while the main limitation lies in how the pipeline is orchestrated between phases. The measured 8.01× speedup on 32 cores is therefore not primarily constrained by poor scalability of the join itself, but by a serialized section that groups histogram merging, prefix-sum computation, per-thread offset construction, and output-buffer allocation. The Amdahl fit ($f \approx 0.101$) is consistent with this interpretation and suggests that further gains will come more from reducing this serialized orchestration cost than from micro-optimising the already well-scaling local join.

A practical consequence is that buffer management deserves particular attention. Moving the allocation outside the timed region, or preallocating and reusing buffers across repeated calls, would directly reduce

the measured serial fraction and should improve both strong and weak scaling.

The performance dip at $T = 20$ is compatible with NUMA-related effects, so binding threads to cores within a single NUMA node (e.g., via `numactl -cpunodebind=0`) is a plausible next experiment for obtaining smoother scaling up to 16 threads. Since this interpretation is inferred from the machine topology and the observed curve, it should be presented as a hypothesis rather than as a confirmed cause.

The partition-count sweep also clarifies the design trade-off. Choosing a larger P improves locality and load balance in the local join, but only up to the point where histogram and scatter overhead begin to offset those gains. In the tested range, $P \geq 512$ is a good compromise, which is consistent with the strong performance of the scatter and join phases in the phase-level breakdown.

7 Conclusions

A parallel partitioned hash join was implemented with C++ threads and `std::barrier`, reusing threads across all pipeline phases. The histogram, scatter, and join phases parallelize effectively; the dominant limitation is the serialized barrier-completion section, which includes both prefix-related bookkeeping and buffer allocation, consistent with Amdahl's model at $f = 0.101$. The implementation is correct for all tested configurations: sequential and parallel outputs match on both `join_count` and the two 64-bit checksums. On 32 cores of node01 the pipeline runs in 294.1 ms for a 60 M-record join, delivering 8.01× speedup over the 2.355 s sequential baseline.